Squire

From Natural Language to Reliable Execution

An AI-Powered Productivity Assistant

Project Report

Team Members

Mohamed Akram Ali Mohamed *(Team Leader)*
Omar Mohamed said mowena
Karim Walaa El-Din Gaber Ali
Omar Alaa Omar Zalat
Ziad Mohamed Ahmed Mohamed
Omar Mohammed Mohammed Ghorab

July 10, 2026

Contents

1 Summary

Squire is an end-to-end, AI-powered personal-productivity assistant that converts plain-language commands (e.g. *"remind me to submit the report tomorrow at 5pm"*) into structured, reliable actions. A custom-trained Natural Language Understanding (NLU) model identifies the user's **intent** (`ADD`, `GET`, `UPDATE`, `DELETE`) and the **object** of that intent (`TASK`, `MEETING`, `NOTE`, `PROGRESS`), while simultaneously extracting entities such as dates, times, people, and locations. The extracted information is validated, disambiguated through a clarification loop when necessary, and executed deterministically against a PostgreSQL database. Where relevant, the system triggers automation workflows (Google Calendar events, email reminders via n8n) and replies to the user in natural language generated by a lightweight local LLM.

The system is composed of four cooperating components: a **custom NLU engine** (fine-tuned mDeBERTa-v3 with adapter layers, exported to quantized ONNX), a **FastAPI backend** that orchestrates post-processing, decision-making, execution, and RAG-based note retrieval, an **n8n automation layer** that integrates with Google Calendar and Gmail, and a **Flutter mobile application** that serves as the user-facing client across Android, iOS, Web, Windows, Linux, and macOS.

Team

Name	Role
Mohamed Akram	System design/NLU/backend/Redis
Omar Mowena	Response layer/database
Karim Walaa	Flutter/n8n
Omar Zalat	RAG (retriever)
Ziad Mohamed	semantic mapper
Omar Ghorab	embedder model

2 Project Overview

Traditional productivity tools (task managers, calendar apps, note-taking apps) typically require users to navigate multiple forms and manually enter structured data — a process that is slow, repetitive, and easily interrupted. Squire addresses this problem by allowing users to interact with the system entirely through natural language, while still guaranteeing safe and deterministic execution of the underlying actions.

The system's request flow is:

Mobile App → *FastAPI* `/predict` → *NLU (ONNX)* → *Postprocessing* → *Decision Engine* → *Executor (PostgreSQL/Redis)* → *n8n Automation* → *LLM Response Generator* → *Mobile App*

2.1 Core Capabilities

- **Custom NLU engine** — a transformer encoder (`microsoft/mdeberta-v3-base`) fine-tuned with lightweight adapter layers for joint intent classification and slot tagging, exported to ONNX (INT8) for fast CPU inference.
- **4 actions × 4 objects** — Add / Get / Update / Delete across Tasks, Meetings, Notes, and Progress entries.
- **Entity extraction** for titles, dates, times, people, locations, statuses, fields/values, and free-text content.

- **Clarification loop** — ambiguous or incomplete commands are tracked in Redis and resolved through a follow-up question instead of a guess.
- **RAG-powered notes** — notes are embedded (`all-MiniLM-L6-v2`) and stored in Postgres + pgvector; questions about notes are answered by retrieving the most relevant chunks and grounding the LLM's response in them.
- **Local LLM response generation** (TinyLlama) for natural replies, note Q&A, and summaries.
- **Automation** — n8n workflows push accepted tasks to Google Calendar and send email reminders ahead of deadlines.
- **Cross-platform mobile client** built with Flutter.

3 Objectives

1. Design and train a custom NLU model capable of joint intent classification, object classification, and named-entity recognition for productivity-domain commands.
2. Build a deterministic decision engine that either executes a validated action or triggers a clarification dialogue when information is missing or ambiguous.
3. Implement a FastAPI backend that connects the NLU pipeline to a PostgreSQL + pgvector database and a Redis-backed conversation-state store.
4. Integrate automation workflows (n8n) that translate accepted actions into real-world effects — Google Calendar events and Gmail reminder notifications.
5. Provide retrieval-augmented generation (RAG) over stored notes so that user questions are answered using grounded, application-specific context rather than the LLM's own memory.
6. Optimize the NLU model for production (ONNX export with INT8 quantization) to enable fast, low-resource CPU inference.
7. Deliver a cross-platform Flutter mobile client offering a chat-style interface, dashboard, calendar view, and note/task library.

4 Project Plan (12 Weeks)

The project was carried out over a 12-week schedule, moving from data collection and model design through backend integration, automation, and final testing.

Weeks	Duration	Phase	Key Activities
1–2	2 weeks	Research & Flutter App	Literature review, NLU architecture research, dataset survey, tool selection, Flutter app scaffolding, screen shells (Calendar, Curator).
3–6	4 weeks	Data & System Design	Dataset schema design, data annotation, dataset audit, system architecture design, database ERD/schema design, API contract design.

Weeks	Duration	Phase	Key Activities
7–8	2 weeks	Database & NLU Model	PostgreSQL/pgvector & Redis setup, DDL implementation, mDeBERTa-v3 + adapters training, phased fine-tuning, evaluation, ONNX export & INT8 quantization.
9–11	3 weeks	RAG, Embedding, Retriever, Decoder & n8n Mapper	Embedding model integration, pgvector retriever, RAG pipeline, TinyLlama decoder, prompt templates, clarification loop, n8n Calendar/Gmail automation mapper.
12	1 week	System Integration	Full end-to-end integration, system testing, bug fixing, UI polish, documentation, presentation preparation.

5 Milestones

Week	Milestone
Week 2	Research phase complete (architecture & dataset review); initial Flutter app scaffold running with navigable screen shells.
Week 6	Custom annotated dataset finalized; system architecture and database ERD/schema design complete.
Week 8	PostgreSQL/pgvector and Redis operational; NLU model fully trained and exported as a quantized ONNX (INT8) model achieving target accuracy (Action: 96.3%, Object: 93.8%, NER Micro-F1: 89.8%).
Week 11	RAG retrieval pipeline, TinyLlama-based response decoder, and n8n automation mapper (Calendar + Gmail) implemented and functioning.
Week 12	Fully integrated system (Flutter app + backend + NLU + RAG + automation) tested end-to-end and demonstrated; final report and presentation delivered.

6 Deliverables

- Custom annotated productivity-command dataset (intents, objects, entities).
- Trained NLU model (PyTorch checkpoint and quantized ONNX INT8 export).
- FastAPI backend source code, including post-processing, decision engine, execution layer, and RAG pipeline.
- PostgreSQL database schema (DDL) with `pgvector` support, and accompanying ERD/documentation.
- n8n automation workflow definitions (`Calendar Tasks.json`, `calendar-reminder-workflow.json`, `Google calendar workflow.json`).
- Cross-platform Flutter mobile application (Dashboard, Calendar, Curator chat, Library screens).
- System architecture diagram and entity-relationship diagram (ERD).

- Project documentation (README files, API reference, setup guides).
- Final project presentation and this project report.

7 Evaluation

The NLU model was evaluated on held-out validation data across its three joint tasks:

Task	Metric	Score
Action Classification	Accuracy	96.3%
Action Classification	Macro F1	96.4%
Object Classification	Accuracy	93.8%
Object Classification	Macro F1	93.9%
Named Entity Recognition (NER)	Micro F1	89.8%
Named Entity Recognition (NER)	Precision	88.1%
Named Entity Recognition (NER)	Recall	91.5%
Reliability	Expected Calibration Error (ECE)	2.04%
Reliability	Embedding Separation	2.75

7.1 Discussion

The model achieves strong accuracy on both intent-related tasks (Action and Object classification), with slightly lower NER performance reflecting the higher difficulty of span-level entity tagging. The low ECE (2.04%) indicates that the model's confidence scores are well-calibrated, which is important for the decision engine's ability to distinguish between confident predictions (safe to execute) and uncertain ones (requiring clarification). Following INT8 quantization, the model showed minimal accuracy loss while gaining substantial improvements in CPU inference speed and memory footprint, validating its suitability for lightweight, local deployment.

The LLM response layer (TinyLlama) was selected after comparing several lightweight local LLMs, balancing response quality against resource consumption suitable for local/CPU deployment.

8 Suggested Tools

Layer	Technology
NLU Model	PyTorch, HuggingFace Transformers, `mdeberta-v3-base` + adapter layers, ONNX Runtime (INT8)
Backend API	FastAPI, Uvicorn, Pydantic
Database	PostgreSQL + `pgvector` (Docker)
Cache / Sessions	Redis (Docker)
Embeddings & RAG	Sentence-Transformers (`all-MiniLM-L6-v2`) + pgvector similarity search
Response Generation	TinyLlama (local, via HuggingFace `transformers` pipeline)
Automation	n8n (Docker) — Google Calendar + Gmail workflows
Mobile Client	Flutter / Dart
Containerization	Docker

9 References

1. He, P., Gao, J., & Chen, W. (2021). *DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing.* arXiv:2111.09543.
2. He, P., Liu, X., Gao, J., & Chen, W. (2020). *DeBERTa: Decoding-enhanced BERT with Disentangled Attention.* arXiv:2006.03654.
3. Hemphill, C. T., Godfrey, J. J., & Doddington, G. R. (1990). *The ATIS Spoken Language Systems Pilot Corpus.* Proceedings of the DARPA Speech and Natural Language Workshop.
4. Coucke, A., et al. (2018). *Snips Voice Platform: An Embedded Spoken Language Understanding System for Private-by-Design Voice Interfaces.* arXiv:1805.10190.
5. Chen, X., et al. (2020). *Low-Resource Domain Adaptation for Compositional Task-Oriented Semantic Parsing (TopV2).* Proceedings of EMNLP.
6. Casanueva, I., Temčinas, T., Gerz, D., Henderson, M., & Vulić, I. (2020). *Efficient Intent Detection with Dual Sentence Encoders (BANKING77).* Proceedings of the 2nd Workshop on NLP for ConvAI.
7. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* Proceedings of EMNLP-IJCNLP. (basis for `all-MiniLM-L6-v2`.)
8. Zhang, P., et al. (2024). *TinyLlama: An Open-Source Small Language Model.* arXiv:2401.02385.